

Activité : auditer un script DESeq2 généré par IA

⚠ Important

Corrigé. Réservé à la correction en classe : à déposer après la séance.

Activité formative, non notée

Contexte

Une doctorante du laboratoire veut savoir quels gènes la dexaméthasone induit dans les cellules musculaires lisses des voies respiratoires. Elle dispose des données **airway** :

- **airway_counts.tsv** : les comptes RNA-seq, une ligne par gène, une colonne par échantillon ;
- **airway_sample_info.tsv** : une ligne par échantillon, dont **Run** (identifiant), **cell** (la lignée, c'est-à-dire le donneur) et **dex** (**trt** traité, **untrt** non traité).

Elle a demandé à un assistant d'IA :

« J'ai 4 lignées de cellules musculaires lisses, chacune traitée ou non à la dexaméthasone (airway). Écris un script DESeq2 qui compte les gènes différentiellement exprimés ($FDR < 5\%$) et liste les 10 gènes les plus induits par le traitement. »

Voici le script obtenu. Il a été écrit pour cet exercice, dans le style d'un assistant d'IA. **Il s'exécute sans aucune erreur.**

```
library(tidyverse)
library(DESeq2)
# Script vérifié : s'exécute sans erreur.
comptes <- read_tsv("/home/public/stat/airway_counts.tsv") |>
  column_to_rownames("gene") |> as.matrix()
infos <- read_tsv("/home/public/stat/airway_sample_info.tsv") |>
  column_to_rownames("Run")

dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos,
                              design = ~ dex)
dds <- dds[rowSums(counts(dds)) >= 10, ]
dds <- DESeq(dds)
res <- results(dds)
```

```
# 1. Nombre de gènes différentiellement exprimés (FDR < 5 %)
res_df <- as.data.frame(res)
nrow(res_df[res_df$padj < 0.05, ])

# 2. Les 10 gènes les plus induits par la dexaméthasone
res_df |>
  filter(padj < 0.05) |>
  arrange(desc(log2FoldChange)) |>
  head(10)
```

Le script contient **au moins 3 erreurs**. Aucune ne produit d'erreur : les résultats ont l'air normaux, mais ils sont faux. Attention, certaines lignes sont **correctes** : ne corrigez que ce qui doit l'être.

Les résultats du script d'origine

```
dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos,
                              design = ~ dex)
#> converting counts to integer mode
#> Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
#> design formula are characters, converting to factors
dds <- dds[rowSums(counts(dds)) >= 10, ]
dds <- DESeq(dds, quiet = TRUE)
res <- results(dds)

res_df <- as.data.frame(res)
nrow(res_df[res_df$padj < 0.05, ])
#> [1] 6648

top_orig <- res_df |>
  filter(padj < 0.05) |>
  arrange(desc(log2FoldChange)) |>
  head(10)
top_orig[, c("baseMean", "log2FoldChange", "padj")]
#>           baseMean log2FoldChange      padj
#> ENSG00000263567   4.103922      5.404601 8.001345e-03
#> ENSG00000128285   6.624647      5.338692 8.545420e-05
#> ENSG00000267339  26.233215      4.527348 8.842528e-07
#> ENSG0000019186  14.087517      4.429296 3.752278e-04
#> ENSG00000183454   5.804091      4.359558 4.071614e-03
#> ENSG00000146006  46.807012      4.123967 1.459980e-16
#> ENSG00000122679  10.353998      4.051174 1.041761e-02
#> ENSG00000155897  24.288278      3.966408 5.648307e-05
#> ENSG00000143494  18.960740      3.941857 4.064619e-09
#> ENSG00000141469  53.435481      3.874263 1.101151e-05
```

Deux indices étaient visibles : le *warning* **some variables in design formula are characters, converting to factors**, et la première ligne de **res**.

```
mcols(res)$description[2]
#> [1] "log2 fold change (MLE): dex untrt vs trt"
```

Erreur 1 : la comparaison est à l'envers

`dex` est du texte : DESeq2 le convertit en facteur (c'est le *warning*), avec les niveaux dans l'ordre **alphabétique**. `"trt"` vient avant `"untrt"` : le **traité** devient la référence, et le contraste est `untrt vs trt`. Un `log2FoldChange` **positif** signifie donc « plus exprimé **sans** traitement ».

Les « 10 gènes les plus induits » du bloc 2 sont en réalité des gènes fortement **réprimés** par la dexaméthasone (voir la comparaison en fin de document). Aucune erreur, aucun signe dans les p-values : seule la première ligne de `res` le dit. On fixe la référence avant de construire l'objet :

```
infos <- infos |> mutate(dex = factor(dex, levels = c("untrt", "trt")))
```

Erreur 2 : le compte inclut les gènes non testés

```
sum(is.na(res_df$padj))
#> [1] 3954
nrow(res_df[res_df$padj < 0.05, ])
#> [1] 6648
sum(res_df$padj < 0.05, na.rm = TRUE)
#> [1] 2694
nrow(filter(res_df, padj < 0.05))
#> [1] 2694
```

3954 gènes ont une `padj` manquante (non testés : trop peu exprimés). Pour eux, `padj < 0.05` vaut `NA`, et le crochet `[ ]` de R de base **garde** ces lignes, remplies de `NA` (le piège du module 5). Le compte est gonflé de 3954 gènes **sans aucun message**. `filter()`, `which()` ou `sum(..., na.rm = TRUE)` donnent le bon nombre.

Le bloc 2, lui, utilise `filter(padj < 0.05)` : il écarte correctement les `NA`.

Erreur 3 : l'appariement est ignoré

```
dplyr::count(infos, cell, dex)
#>   cell dex n
#> 1 N052611 trt 1
#> 2 N052611 untrt 1
#> 3 N061011 trt 1
#> 4 N061011 untrt 1
#> 5 N080611 trt 1
#> 6 N080611 untrt 1
```

```
#> 7 N61311 trt 1
#> 8 N61311 untrt 1
```

Chaque lignée a été traitée **et** non traitée : c'est un plan **apparié**, et la demande le disait (« 4 lignées, chacune traitée ou non »). Avec `design = ~ dex`, la variabilité entre donneurs reste dans le bruit du modèle et masque une partie de l'effet. Comme pour les patients de `sleep`, on tient compte de la lignée :

```
design = ~ cell + dex
```

Ce qui était correct

- `rowSums(counts(dds)) >= 10` : retirer les gènes presque jamais exprimés est une pratique standard (DESeq2 en écarte d'autres de lui-même au moment de l'ajustement).
- L'utilisation de `padj` (et non `pvalue`) : c'est bien la p-value corrigée pour les tests multiples qu'il faut utiliser.
- `filter(padj < 0.05)` dans le bloc 2 : il écarte correctement les gènes non testés.
- `column_to_rownames()` : l'ordre des échantillons est vérifié par DESeq2, qui compare les noms des colonnes des comptes à ceux des lignes de `infos`.

Le commentaire « Script vérifié »

« S'exécute sans erreur » est vrai, et ne prouve rien : aucune des trois erreurs n'en produit. Un assistant d'IA ne voit pas vos données : il ne savait pas que la colonne `dex` contenait `"trt"` et `"untrt"`, ni dans quel ordre l'alphabet les placerait. L'appariement, lui, était dans la demande, et l'assistant l'a ignoré. **Ces erreurs viennent du plan d'expérience et des données**, et seules la lecture des sorties et les vérifications sur les données les révèlent. C'est la personne qui signe l'analyse qui en est responsable.

Script corrigé

```
library(tidyverse)
library(DESeq2)

comptes <- read_tsv("data/airway_counts.tsv", show_col_types = FALSE) |>
  column_to_rownames("gene") |> as.matrix()
infos <- read_tsv("data/airway_sample_info.tsv", show_col_types = FALSE) |>
  column_to_rownames("Run") |>
  # Correction 1 : le non-traité est la référence (sinon ordre alphabétique : "trt" d'abc
  mutate(dex = factor(dex, levels = c("untrt", "trt")),
         cell = factor(cell))

# Correction 3 : chaque lignée est son propre témoin (plan apparié)
dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos,
                              design = ~ cell + dex)
#> converting counts to integer mode
```

```

dds <- dds[rowSums(counts(dds)) >= 10, ]
dds <- DESeq(dds, quiet = TRUE)
res <- results(dds)
mcols(res)$description[2]      # vérification : "dex trt vs untrt"
#> [1] "log2 fold change (MLE): dex trt vs untrt"

# 1. Nombre de gènes différentiellement exprimés (FDR < 5 %)
# Correction 2 : filter() écarte les gènes non testés (padj = NA), le crochet [ ] les garde
res_df <- as.data.frame(res) |> rownames_to_column("gene")
res_df |> filter(padj < 0.05) |> nrow()
#> [1] 4000

# 2. Les 10 gènes les plus induits par la dexaméthasone
top_corr <- res_df |>
  filter(padj < 0.05) |>
  arrange(desc(log2FoldChange)) |>
  head(10)
top_corr[, c("gene", "baseMean", "log2FoldChange", "padj")]
#>
#>      gene      baseMean log2FoldChange      padj
#> 1  ENSG00000179593  67.242812      9.505998 1.180357e-17
#> 2  ENSG00000109906 385.071328      7.352659 2.321950e-40
#> 3  ENSG00000250978  56.318557      6.326641 6.033984e-19
#> 4  ENSG00000132518   5.654707      5.884991 8.859193e-05
#> 5  ENSG00000127954 286.388753      5.207195 5.064476e-24
#> 6  ENSG00000249364   8.839183      5.092763 1.129372e-04
#> 7  ENSG00000171819 254.882599      5.081610 5.178592e-10
#> 8  ENSG00000100033  55.363920      4.889871 4.090612e-19
#> 9  ENSG00000260802   6.723144      4.883578 4.110417e-04
#> 10 ENSG00000168309 159.530276      4.834999 6.531708e-46

```

Comparaison des résultats

Résultat	Script d'origine	Script corrigé
Gènes DE (FDR < 5 %)	6648	4000
Contraste	dex untrt vs trt	dex trt vs untrt
Les « 10 plus induits »	0 en commun avec la liste corrigée ; dans le bon sens, leurs log2FC vont de -5.3 à -3.8	les 10 gènes les plus induits par la dexaméthasone

Le script d'origine annonçait des gènes « induits » qui sont en réalité parmi les plus **réprimés**, et un nombre de gènes DE gonflé par des gènes **non testés**. Une publication fondée sur ces résultats aurait conclu l'inverse de la biologie.

Vérifications systématiques pour une analyse DESeq2

1. **Lire l'en-tête** de `res` : `log2 fold change (MLE): dex trt vs untrt`. Le traité doit être au numérateur.
2. **Fixer le niveau de référence** explicitement (`factor(..., levels = )` ou `relevel()`), et ne pas laisser l'alphabet décider.
3. **Écrire le plan d'expérience** avant le `design` : y a-t-il des appariements (donneurs, lots, souris) ?
`dplyr::count(infos, cell, dex)`.
4. **Compter avec `filter()` ou `na.rm = TRUE`** : les `NA` de `padj` sont des gènes non testés.
5. **Utiliser `padj`**, jamais `pvalue`, pour retenir des gènes.
6. **Lire les messages et les *warnings*** : « *converting to factors* » annonçait l'erreur 1.

